SOL

Programming Language

Language and Toolchain Design Specification

Concept Design v0.2

August 9, 2026

A LANGUAGE OPTIMIZED FOR HUMAN-AI CO-DEVELOPMENT

STATUS

Concept Draft with Executable-Core Decisions

Target language design; experimental bootstrap front end; not a standardized or production-ready language.

Document Control

Field	Value
Document	Sol Programming Language - Language and Toolchain Design Specification
Version	0.2 Concept Draft
Date	August 9, 2026
Status	Concept Draft with Executable-Core Decisions
Baseline	Target design updated through the current executable-core checkpoint
Primary objective	Define a language whose semantics, tooling, and source representation optimize safe maintenance by humans and AI systems.
Normative vocabulary	MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY distinguish required, recommended, and optional behavior.
Authority	`docs/specification.typ` is authoritative. The root v0.2 PDF is generated from it.

SCOPE This document specifies a coherent target design, not merely a feature wishlist. It also records decisions already executable in the bootstrap front end. Unless a passage is explicitly marked **IMPLEMENTED**, it describes target language behavior. Items marked **FUTURE WORK** are not implemented.

Executive Summary

Sol is a statically typed, memory-safe, effect-aware, verification-capable programming language designed around the reality that software is continuously modified by teams containing both humans and generative models. Its central optimization target is not keystroke count. It is **preserving intent and correctness while changing code**.

The target language combines a Rust-like safe implementation substrate, Koka/F*-style effect tracking, Dafny-style contracts, algebraic data types, explicit capabilities, structured concurrency, state-machine protocols, schema-aware data evolution, and a stable semantic representation for programmatic editing. It intentionally avoids unrestricted textual metaprogramming, unchecked exceptions, implicit nullability, ambient authority, and multiple equivalent syntactic idioms.

Sol uses a two-tier correctness model. The base language provides fast, predictable compilation with nominal types, ownership, exhaustive matching, explicit effects, and typed errors. A proof layer adds refinements, preconditions, postconditions, invariants, ghost state, and SMT-backed obligations. Normal application code need not become a theorem-proving exercise, but critical boundaries can be strengthened without changing languages.

The compiler is intended to expose a canonical typed semantic graph, stable declaration identities, machine-readable diagnostics, constrained repair actions, proof caches, and behavior-change reports. These interfaces let tools patch declarations and obligations rather than brittle line numbers. Humans retain authority over intent, architecture, accepted effects, and proof policy.

> IMPLEMENTED The C17 bootstrap is an experimental edition-2027 compiler front end with a library-only reference interpreter, not a production executor. It currently lexes and parses a core language, resolves lexical, multi-file module, explicit import, and declaration-owned type/effect names, assigns stable top-level semantic IDs and resolved occurrence records, checks built-in, nominal, bounded first-order generic, and bounded trait semantics, exhaustive matches, closed normalized effects plus callback-driven rank-1 row parameters, capabilities and exact handlers, lowers deterministic contract templates, produces owning deterministic typed IR, and evaluates its bounded immutable core after frontend teardown. Ownership, richer traits and general constrained or recursive row polymorphism, runtime contract checks, `sol test`, `sol run`, SMT discharge, code generation, stable public schemas/IR, and semantic patches remain future work.

Key Decisions

Area	Decision
Execution	Ahead-of-time native compilation is primary; WebAssembly components and a lightweight VM/REPL are first-class secondary targets.
Memory	Affine ownership with inferred borrowing and lexical/explicit regions; ordinary code requires no tracing garbage collector.
Types	Nominal algebraic types, distinct newtypes, generics, traits, refinements, units, and limited dependent relationships.
Effects	The target is general row polymorphism; the bootstrap implements closed normalized finite rows, one callback-inferred free-function row parameter, and exact capability-sensitive effects.
Failure	Typed `Result` for recoverable failures; `panic` is a declared bug effect and forbidden in verified profiles.
Verification	Contracts generate obligations. Bootstrap templates are implemented; runtime checks, abstract interpretation, SMT, and proof discharge are future work.
Concurrency	Structured task scopes, cancellation propagation, race-safe sharing, actors/channels, and protocol-typed resources.
Metaprogramming	Typed derives and semantic transforms only; no unrestricted token macros in the safe language.
AI interface	Stable semantic IDs, canonical IR, structured diagnostics, semantic patches, and change-oriented compilation.

Area	Decision
Compatibility	Explicit versioned schemas, API fingerprints, migrations, and capability-aware FFI boundaries.

Contents

READING PATH Readers evaluating the concept should begin with Sections 1, 2, 5, 6, and 9. Compiler implementers should continue through Sections 10 and 12. Section 12.2 and the status callouts distinguish the executable bootstrap from the target design. Worked examples are normative enough to expose proposed syntax inconsistencies, but only examples explicitly labeled bootstrap are expected to compile today.

1 Vision, Scope, and Design Principles

1.1 Problem Statement

Most mainstream languages were designed around a human author entering text and a compiler deciding whether it can execute. Modern development has a different bottleneck. Large programs are maintained through repeated modifications under incomplete context, often across repositories, schemas, services, and toolchains. Generative models amplify both the rate of change and the cost of ambiguity.

The most damaging errors rarely reflect an inability to express an algorithm. They fail to preserve hidden assumptions: whether a value can be absent, a function performs I/O, an operation is atomic, a resource escapes, a retry is idempotent, an old reader accepts a new schema, or a refactor alters observable behavior. Conventional languages often leave these facts in comments, naming, framework behavior, or tribal knowledge.

Sol treats these facts as language semantics. Inputs and outputs alone do not describe a function; declared effects, errors, ownership, contracts, cost constraints, and protocol transitions are also interface. The compiler maintains a semantic graph tools can query without reconstructing meaning from text.

1.2 Product Vision

Sol should support ordinary services, systems tools, embedded control, libraries, and WebAssembly components without requiring formal-methods expertise. The same codebase should let critical modules add stronger contracts and proofs incrementally. Verification is a gradient, not a separate language or all-or-nothing project decision.

The intended loop is: state intent and constraints; implement inside a narrow semantic envelope; compile to receive structured obligations; apply a semantic change; inspect a behavior-change report. The safe, reviewable path should be easier than the clever path.

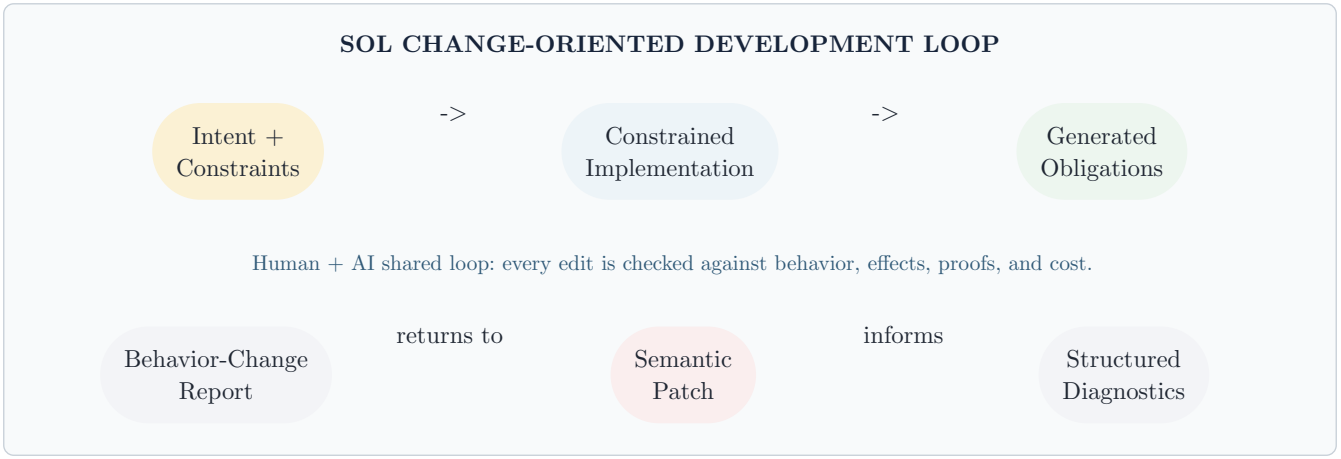

Figure 1: Sol optimizes the change loop, not merely the authoring step.

1.3 Primary Goals

- **Local semantic clarity.** A declaration reveals nullability, mutation, effects, failure, ownership, and contracts without distant configuration.
- **Invalid-state resistance.** Types represent domain states, protocols, and units rather than correlated booleans and comments.
- **Change safety.** Tooling reports behavior, effect, schema, cost, and proof changes caused by a patch.
- **AI-legible structure.** The compiler exposes canonical identities, structured diagnostics, and valid repair classes.
- **Progressive verification.** Ordinary code compiles quickly; high-assurance code states and proves stronger properties.
- **Predictable performance.** Zero-cost abstractions, deterministic destruction, bounded allocation profiles, and visible low-level escape hatches remain possible.
- **Interoperability.** Sol enters existing systems through C, WebAssembly components, generated bindings, and schema-defined services.
- **Canonicality.** Formatting and syntax minimize semantically meaningless variation.

1.4 Non-Goals

- Sol does not maximize terseness, code-golf expressiveness, or syntactic customization.

- Sol 1.0 will not expose unrestricted dependent types throughout ordinary programs.
- Sol does not guarantee every terminating program can be proved automatically; annotations, lemmas, or runtime checks may be required.
- Sol does not hide distributed-system failure behind transparent remote objects.
- During pre-1.0 work, source compatibility does not override semantic clarity.
- Safe code cannot acquire filesystem, network, process, clock, random, database, or unsafe-memory authority implicitly.

1.5 Design Principles

Principle	Consequence
Intent is part of the program	Contracts, effects, protocols, schema policy, and preservation constraints are versioned beside implementation.
Public interfaces are explicit	Inference reduces local annotation, but exported types and effects have one normalized form.
One obvious form	The formatter is mandatory; alternate ordering, brace, import, and equivalent sugar are minimized.
Capability before convention	I/O and authority require values or declared effects, never ambient global access.
Proofs must compose	Verification is modular, cached, and bounded by interfaces; whole-program SMT is not normal.
Escape hatches are visible	`unsafe`, unchecked FFI, raw memory, nondeterminism, and assumptions are explicit and auditable.
Diagnostics are an API	Compiler output is structured, stable, and carries machine-actionable repair alternatives.
Source is not the sole truth	A canonical typed semantic graph is a supported artifact, not an implementation accident.

1.6 Target Users and Domains

First-class domains are backend services, command-line tools, systems components, embedded and real-time control, security-sensitive libraries, data pipelines, and WebAssembly components. They reward explicit effects and predictable resources while sharing enough abstractions for one language.

The initial language is not optimized for browser UI frameworks, highly dynamic scripting, symbolic mathematics, or GPU kernels, though it should interoperate with them. A managed profile may improve desktop ergonomics later, but the baseline object model cannot depend on tracing collection.

2 Language Overview and Canonical Source Model

2.1 A Small Complete Example

```
module inventory.reservation
use commerce.OrderId
use inventory.{ItemId, Quantity}
use time.Instant

enum ReservationError {
    item_not_found(item: ItemId)
    insufficient_stock(item: ItemId, requested: Quantity, available: Quantity)
}

record Reservation {
    order: OrderId
    item: ItemId
    quantity: Quantity where quantity > 0
    created_at: Instant
}

public transaction reserve(
    order: OrderId,
    item: ItemId,
    quantity: Quantity where quantity > 0,
) -> Result<Reservation, ReservationError>
effects { database.transaction<Inventory> clock.read }
ensures {
    success => inventory[item].available
        == old(inventory[item].available) - quantity
    failure => inventory[item].available
        == old(inventory[item].available)
} {
    let stock = inventory.lock(item)
        .or_error(ReservationError.item_not_found(item))?
    require stock.available >= quantity else {
        return ReservationError.insufficient_stock(
            item = item, requested = quantity, available = stock.available,
        )
    }
    stock.available -= quantity
    return Reservation { order, item, quantity, created_at = clock.now() }
}
```

Listing 1: Representative target-design declaration with domain types, effects, a transaction boundary, and postconditions.

Modules and imports are explicit. Data types are algebraic and nominal. Nullability is `Option<T>`; recoverable failure is `Result<T, E>`. Transaction authority is visible. Contracts attach to declarations. Mutation is confined to a compiler-known resource scope. Transactions, refinements, imports, and this full example remain target design rather than accepted bootstrap input.

2.2 Lexical and Formatting Rules

- UTF-8 source is supported; public identifiers SHOULD use a restricted profile against confusables.
- Keywords and standard-library identifiers are lowercase ASCII. Types use UpperCamelCase; values, fields, modules, and effects use snake_case.
- Blocks use braces. Semicolons are absent. Canonical newlines separate statements.
- Indentation is four spaces; canonical formatting rejects tabs.
- Multiline arguments, fields, and variants require trailing commas; single-line forms omit them.
- One formatter produces stable output for each edition and defines canonical source representation.
- Comments use `//`, `/* ... */`, and `///`; nested block comments are supported.

CANONICAL SOURCE The compiler accepts a narrow grammar and `sol fmt` produces a normalized spelling. Canonicality reduces diff noise, simplifies generated patches, and stabilizes semantic hashes.

IMPLEMENTED The bootstrap formatter accepts syntactically valid edition-2027 files and deterministic directory packages. It preserves source token/comment bytes, source order, commas, and deliberate hard line breaks while normalizing spaces, four-space indentation, LF line endings outside comments, blank-line runs, final newline, braces, operators, delimiters, generic/effect angles, and comparisons. Output is re-lexed, reparsed, checked for token preservation, and formatted again for byte idempotence. `sol fmt` writes through a staged package transaction; `--check` is read-only and `--stdout` accepts one file. Malformed files are never rewritten. Width-based reflow, trailing-comma insertion/removal, import/declaration sorting, and comment/semantic reordering are future work.

2.3 Declaration Ordering

Canonical order is module; edition/features; imports; public types; private types; constants; capabilities; functions; protocol/transaction declarations; specifications; tests. Tools MAY preserve deliberate named regions, but arbitrary ordering is not a style preference.

```
module billing.transfer
edition 2027
use identity.AccountId
use money.Money
public enum TransferError { ... }
public record TransferReceipt { ... }
const maximum_transfer = 100_000 USD
public function transfer(...) -> ...
effects { ... }
requires { ... }
ensures { ... } {
    ...
}
spec transfer { ... }
test "same-account transfers fail" { ... }
```

Listing 2: Canonical target file structure.

2.4 Expressions and Statements

Sol is expression-oriented but distinguishes effectful statements where reviewability improves. `if`, `match`, and blocks produce values. Assignment, resource acquisition, spawning, transaction control, and capability operations are statements. A block's final expression is its value.

```
let label = match account.state {
    active => "Active"
    frozen(reason) => "Frozen: {reason}"
    closed(at) => "Closed at {at}"
}
let fee = if amount >= 1_000 USD { 0 USD } else { 2 USD }
```

Listing 3: Exhaustive matching and expression-valued control flow.

2.5 Bindings and Mutation

- `let` creates an immutable binding.
- `var` creates a mutable local and is prohibited in pure-expression contexts.
- Records are immutable unless accessed through an exclusive mutable borrow or owned `modify` scope.
- Mutation syntax remains ordinary once exclusivity is proven.

```
let original = account
modify account {
    account.balance += deposit
    account.updated_at = clock.now()
}
// `original` is an immutable value, not an alias to modified storage.
```

Listing 4: Explicit mutation scope.

2.6 Functions and Methods

Free functions are primitive. Method syntax is type-directed sugar when the first parameter is `self`. Extension methods are namespaced imports, not global monkey patches. Functions are nonvirtual except through trait values. Overload resolution is deliberately limited.

```
public function Money.add(
    self: Money<Currency>, other: Money<Currency>,
) -> Money<Currency>
effects { pure } {
    return Money(self.minor_units + other.minor_units)
}
let total = subtotal.add(tax)
```

Listing 5: Method syntax as a projection of an ordinary function.

2.7 Modules and Visibility

A module is a semantic namespace and compilation unit, not necessarily one file. Packages define a public module tree. Visibility is private, module, package, or public. Friend access is absent; cross-package internal access requires an explicit capability or separately versioned package.

Imports never globally change method lookup. Wildcards are rejected in library source and permitted only in REPL/test scopes. External names in public signatures carry canonical fully qualified identities in the semantic graph even when source uses imports.

> **IMPLEMENTED** For directory `sol check`, the bootstrap recursively discovers regular `.sol` files in deterministic bytewise path order as one dependency-free package. Each file declares its module, multiple files may contribute to one module, and private declarations are visible throughout that module. File-scoped `use module.path.Symbol` imports resolve one public declaration and feed package-wide type, generic-bound, trait, effect, and contract checking. Import cycles are allowed because the subset has no top-level initialization. Direct file checking retains the earlier ignored-import compatibility behavior. Manifests, external dependencies, aliases, grouped/wildcard/relative imports, re-exports, separate private/module/package visibility, imported extension-method search, and stable cross-build identities are future work.

3 Type System and Data Modeling

3.1 Type-System Strategy

The target uses a layered static type system. A decidable base provides nominal algebraic types, generics, traits, affine ownership, effect rows, and exhaustiveness. Refinements add predicates over values and state. Proof terms, propositions, lemmas, and solver-guided obligations form a proof layer. Crossing to weaker layers requires erasure or a verified boundary.

USABILITY DECISION Routine syntax does not expose unrestricted full-spectrum dependent typing. Value-dependent constraints are admitted where obligations and tooling remain clear; advanced propositions stay in the proof sublanguage.

IMPLEMENTED The bootstrap implements first-order type parameters on records, enums, and free functions plus one trailing effect-row parameter on free functions. Parameters are rigid declaration-owned identities while templates are checked. Built-in and user applications are invariant and canonically interned by constructor identity plus ordered exact arguments; `Option<T>` and `Result<T, E>` retain their exact one- and two-argument identities. Type positions require complete explicit application. Function calls use either all explicit type arguments or argument-only recursive structural inference; callback arguments infer the bounded effect parameter. Free-function type parameters may carry one inline trait bound, checked after inference and forwarded symbolically through bounded generic calls, including across modules in one checked directory package. Defaults, variance, higher-kinded/lifetime/const parameters, multiple bounds, general effect constraints, generic capabilities/members, polymorphic recursion, separately compiled dependency instantiation, and execution/monomorphization are future work.

3.2 Primitive Types

Category	Types and behavior
Integers	`Int8` through `Int128`, unsigned equivalents, pointer-sized `IntSize`/`UIntSize`, and arbitrary `BigInt`; overflow policy is explicit.
Floating point	`Float16`, `Float32`, and `Float64` with IEEE-oriented semantics and explicit NaN behavior.
Decimal	Fixed-scale `Decimal<Scale>` and arbitrary decimal; no implicit binary-float conversion.
Boolean	`Bool`; domain booleans are discouraged when enums communicate meaning.
Text	`Text` is immutable Unicode scalar text; `Bytes` is immutable bytes; integer text indexing is unsupported.
Characters	`Rune` is a Unicode scalar; grapheme operations require explicit locale/data dependencies.
Never/unit	`Never` has no values. `Unit` has one and prints `()`.

Only `Int64`, `Bool`, `Text`, `Unit`, and `Never` from this primitive catalog currently participate in bootstrap expression typing.

3.3 Records, Enums, and Tuples

Records are nominal products. Enums are nominal tagged unions with typed payloads. Matching a closed enum is exhaustive. Public enums are closed by default; an explicitly open enum admits unknown future variants and requires a fallback. The bootstrap substitutes rigid type parameters through generic record construction/access and generic enum construction/pattern bindings.

```
record Address { street: Text, city: Text, region: Text, postal_code: PostalCode }
enum PaymentResult {
    approved(receipt: Receipt)
    declined(reason: DeclineReason)
    retryable(error: GatewayError, after: Duration)
}
open enum WireMessage { request(Request) response(Response) }
```

Listing 6: Nominal products, closed sums, and opt-in extensible wire enums.

Tuples are structural and local; public returns with more than two independently meaningful fields SHOULD use named records. The bootstrap checks records, enums, constructors, closed/open exhaustiveness, `Bool` matches, and unreachable/duplicate arms; tuples are future work.

3.4 Distinct and Refined Types

A distinct type shares representation but not implicit interchangeability. A refined type carries a predicate. Construction generates a proof obligation or runtime validation according to profile.

```
type UserId = distinct Uuid
type AccountId = distinct Uuid
type Age = refined Int where 0 <= self <= 150
type EmailAddress = refined Text using email.is_valid
type NonEmpty<T> = refined List<T> where self.length > 0
```

Listing 7: Domain identity and value refinement.

Widely instantiated predicates must be solver-friendly and effect-free. Refinements do not alter representation unless a validation tag or witness is retained.

IMPLEMENTED The bounded bootstrap supports generic and nongeneric `type Name = distinct Representation` declarations as fresh nominal identities with explicit one-argument construction and no implicit representation conversion. `type Name = refined Representation where predicate` binds contextual `self` at the representation type, requires `Bool`, applies finalized contract-purity rules, and lowers one deterministic type-owned obligation template. Generic declarations are checked once under rigid parameters and generic distinct construction requires explicit arguments. Public declarations import across package modules, and exact closed declared types may be trait implementation targets. Capability-containing or cyclic representations are rejected. The reference interpreter deliberately ignores contracts and does not construct or validate refined values; predicate assumptions, runtime validation, proof discharge, `using` predicates, inline refinements, projection/destructuring, and refinement-aware exhaustiveness remain future work.

3.5 Optionality and Absence

There is no implicit null. `Option<T>` has `some(T)` and `none`. Nullable foreign pointers must be wrapped, checked, or represented by a foreign handle.

```
let nickname: Option<Text> = user.nickname
match nickname {
    some(value) => greet(value)
    none => greet(user.display_name)
}
```

Listing 8: Explicit absence.

3.6 Generics, Traits, and Constraints

Generics are reified in the semantic graph and normally monomorphized natively. Backends MAY use dictionaries or shared instantiations for ABI/code-size policy. Conformance is explicit and coherent: a package implements a trait for a type only if it owns the trait or type.

```
trait Display {
    function display(self: Self) -> Text
    effects { pure }
}
implementation Display for Int64 {
    function display(self: Self) -> Text effects { pure } {
        return "number"
    }
}
public function render<T: Display>(value: T) -> Text
effects { pure } {
    return value.display()
}
```

Listing 9: Trait-bounded generic function.

Traits may declare associated types, constants, and effects in the target language. Trait objects require explicit `dynamic Trait` and define an ABI-stable dispatch boundary. Arbitrary overlapping specialization is absent.

IMPLEMENTED The bounded bootstrap supports nongeneric traits, exact closed implementations, one inline bound per free-function type parameter, and immediate type-directed dot calls. Requirements and implementation methods use `self: Self`, explicit canonical parameter/result types, and exactly matching closed authority-independent effect rows. One coherent implementation provides every required method exactly once. Concrete calls select an exact implementation; symbolic calls on rigid parameters select their declared requirement, and checked metadata carries that choice into effect and contract purity validation. Generic traits and methods, defaults, method contracts or authority clauses, generic/blanket/conditional implementations, associated items, inheritance, multiple bounds, method values, trait objects, specialization, imported extension lookup, and dependent method effects remain future work.

3.7 Units, Dimensions, and Currency

Physical dimensions and currencies participate in numeric types. Dimensional algebra rejects incompatible addition or comparison. Conversions are explicit and can be zero-cost for known ratios.

```
let distance: Meter = 100 meters
let elapsed: Second = 9.58 seconds
let speed: Meter / Second = distance / elapsed
function kinetic_energy(mass: Kilogram, velocity: Meter / Second) -> Joule {
    return 0.5 * mass * velocity^2
}
let invoice_total: Money<USD> = 125.00 USD
let euro_total = exchange.convert(invoice_total, to = EUR)?
```

Listing 10: Dimensional analysis and currency identity.

Dimension metadata erases when representation is unambiguous. Currency conversion is effectful because rates depend on source and timestamp. Units are not implemented.

3.8 Collections and Views

Type	Semantics
`Array<T, N>`	Fixed contiguous owned array; `N` may be a compile-time natural.
`Vector<T>`	Growable contiguous owned collection.
`List<T>`	Persistent immutable list optimized for structural sharing.
`Map<K, V>`	Concrete ordered or hashed maps expose ordering guarantees.
`View<T>`	Read-only borrowed sequence with lifetime inferred from its source.
`Slice<T>`	Contiguous borrowed range; mutable slices require exclusivity.
`Stream<T, E>`	Effectful asynchronous sequence with visible effect and error types.

Operations distinguish eager allocation, lazy iteration, and effectful streaming. A transformation cannot silently become allocating without affecting a cost contract or change report.

3.9 Pattern Matching and Exhaustiveness

Patterns destructure records, enums, tuples, refined values, and protocol states. Guards are pure. Exhaustiveness accounts for closed variants and simple refinements; harder coverage may become a proof obligation. Wildcards on closed enums are discouraged because they hide new variants.

3.10 Type Inference Boundaries

The target infers locals, generic arguments, borrow regions, and effect rows when unambiguous. Public declarations, trait members, recursive functions, FFI, serialized fields, and persistent schemas require canonical types. The bootstrap requires declared parameter and return types; its private function effect inference, including recursive functions, is described in Section 5.

4 Ownership, Regions, and Resource Safety

4.1 Ownership Model

Every nontrivial resource has an owner. Values are affine by default: consumed at most once unless `Copy` or borrowed. Moving invalidates the source. Destruction is deterministic at scope exit unless ownership returns, transfers, or enters a longer-lived region.

The surface aims to expose less lifetime syntax than Rust while retaining comparable safety. Most borrow regions derive from lexical structure, effect boundaries, and escape analysis. Explicit regions appear where an API relates input and output lifetimes.

```
function first<'data, T>(values: View<'data, T>) -> Option<View<'data, T>>
effects { pure } {
    ...
}
```

Listing 11: Explicit region only where output borrows from input.

> **FUTURE WORK** Affine moves, copying, use-after-move diagnostics, borrows, regions, deterministic cleanup, unsafe boundaries, and FFI rules are target design and are not checked by the bootstrap.

4.2 Borrow Categories

Form	Meaning
`View<T>` or `borrow T`	Shared immutable borrow; any number coexist while mutation is excluded.
`inout T`	Exclusive mutable borrow for a call or scope.
`Owned T`	Callee receives responsibility for destruction or transfer.
`shared T`	Explicit reference-counted/runtime ownership; traits govern task safety.
`handle<Resource>`	Opaque capability-bearing resource with protocol operations and deterministic close.

4.3 Regions and Arenas

Lexical regions permit bulk allocation/destruction. Region values cannot escape unless independent or moved/copied into a longer-lived owner. Region allocation is a standard capability and may be forbidden in heap-free profiles.

```
region frame {
    let vertices = Vector<Vertex>.with_capacity(mesh.vertex_count, in = frame)
    build_vertices(mesh, into = vertices)
    renderer.draw(vertices.view())
}
// All frame allocations are reclaimed here.
```

Listing 12: Region-scoped allocation.

4.4 Destructors and Cleanup

`Drop` provides deterministic cleanup. Cleanup cannot report a recoverable error because another failure may be active. Fallible shutdown uses `close() -> Result<(), CloseError>`; `Drop` performs only best-effort nonblocking safety cleanup. Destructors have tightly constrained effects: network, arbitrary blocking, spawning, and visible logging require unsafe/profile permission.

4.5 Interior Mutability and Sharing

Interior mutability uses explicit synchronization/cell types with visible effects. Safe cross-task sharing requires `Share`; transfer requires `Send`. Compiler-governed primitive ownership traits may be manually implemented only with proof or `unsafe`.

```
let cache: shared RwLock<Map<Key, Value>> = RwLock.new({})
function lookup(key: Key) -> Option<Value>
effects { sync.lock<cache> } {
    using read = cache.read()
    return read.get(key).copied()
}
```

Listing 13: Synchronization represented as authority and effect.

4.6 Unsafe Code

`unsafe` is an effect and lexical block. It permits raw pointers, unchecked refined values, unchecked FFI, or asserted ownership facts. Every block declares assumed invariant and established safe property.

```
unsafe because {
    assume buffer points to `length` initialized bytes
    establish returned slice is valid for region `packet`
} {
    return Slice.from_raw_parts(buffer, length, in = packet)
}
```

Listing 14: Unsafe proof boundary.

Unsafe blocks enter the semantic graph and audits; changed assumptions invalidate dependent proof caches.

4.7 Real-Time and Allocation Profiles

Packages/functions may declare `general`, `bounded`, `heap_free`, or `real_time`. Profiles constrain allocation, blocking, synchronization, panic, and destructor behavior and compose through effects and cost contracts.

```
function control_tick(state: inout ControllerState, input: SensorFrame) -> ActuatorFrame
effects { pure }
resources {
    profile = real_time
    stack <= 2 KiB
    heap == 0 B
    worst_case_time <= 200 us
} {
    ...
}
```

Listing 15: Resource-constrained function.

5 Effects, Capabilities, and Failure Semantics

5.1 Why Effects Are in Function Types

Inputs and outputs do not reveal clock, randomness, database, network, locking, suspension, allocation, logging, panic, or unsafe behavior. These affect determinism, security, testing, retries, caching, and cost. Every function computation type therefore has an effect row.

Pure functions are the simplest case. An effectful function declares a normalized set. Local inference is permitted, but exported signatures and trait requirements print effects explicitly. A caller must possess compatible capability authority or handle the effect.

```
function calculate_total(cart: Cart, rules: PricingRules) -> Money<USD>
effects { pure }
function submit_order(order: PendingOrder) -> Result<OrderId, SubmitError>
effects {
    database.transaction<Orders>
    network.call<PaymentGateway>
    clock.read
    random.secure
}
```

Listing 16: Pure and effectful target function types.

5.2 Effect Rows and Polymorphism

The target design has open, normalized, row-polymorphic effects. Ordering is canonical and does not imply execution order.

```
function map<T, U, effects E>(
    values: View<T>,
    transform: function(T) -> U effects E,
) -> Vector<U>
effects { E memory.allocate }
```

Listing 17: Target effect-polymorphic higher-order function.

> **IMPLEMENTED** The bootstrap implements closed normalized rows plus a bounded, rank-1 row-polymorphic subset. A free function may declare one trailing `effects E` parameter, use it in structural callback parameter types, and include it in its explicit row. Calls infer the least closed authority-free argument from callbacks, currently containing only `panic` and `diverge`, union constraints across occurrences, and publish a deterministic instantiated call row. `pure` and `{}` are empty; private omitted rows still use SCC least-fixed-point inference. Explicit or multiple row arguments, all result-position row parameters, authority-dependent row capture, recursive row polymorphism, generic function callback coercion, and handler transformation of unresolved rows remain unsupported.

Recursive private inference computes a least fixed point over call-graph strongly connected components. Formal capability arguments and `Self` effects substitute at calls, including aliases and mutual/self recursion. Explicit rows are modular boundaries but are checked against every performed effect.

The bootstrap normal form distinguishes authority-free atoms, capability-parameter atoms, and member-relative `Self` atoms. Effect rows disclose behavior but do not grant permission. Exact unparameterized `panic` and `diverge` are the only compiler-defined authority-free atoms. Every other non-pure executable atom requires a lexical capability parameter, or `Self` on a capability member. Unparameterized resource effects and static-path authorities are rejected because the bootstrap has no package-global or runtime static authority root; importing a name never adds authority. Closed structural callback rows and inferred row arguments may contain only authority-free atoms because callback types cannot capture lexical capability provenance. This closes ambient static authority in the checked subset, but host wiring, unsafe/FFI gates, package policy, and runtime sandbox enforcement remain future work.

5.3 Capabilities

An effect states what may happen; a capability value states which authority enables it. Capabilities can be passed, restricted, wrapped, and replaced in tests. Application wiring supplies authority; importing a module cannot create it.

```
capability PaymentGateway {
    function authorize(request: AuthorizationRequest)
        -> Result<Authorization, GatewayError>
    effects { network.call<Self> }
}
function checkout(gateway: capability PaymentGateway, order: Order)
    -> Result<Receipt, CheckoutError>
effects { network.call<gateway> } {
    ...
}
```

Listing 18: Capability-bearing service boundary.

5.3.1 Authority roots and may-origin sets

IMPLEMENTED Each capability value and bound operation carries an interned, sorted, duplicate-free, finite nonempty set of possible lexical capability-parameter roots. A runtime value still has one root; this set is conservative may-origin provenance. Immutable aliases, blocks, exact authority-preserving calls, restrictions, and wrappers preserve it. Reachable `if` and `match` branches union their sets; `Never` branches contribute nothing. Type and effect checking validate canonical root sets defensively.

For a parameter- or `Self`-dependent operation, the bootstrap conservatively expands one ordinary parameterized effect atom for every possible root. Explicit rows must contain every expansion; inferred rows union them, including recursively. This is finite closed-row expansion, not a row variable. Mixed provenance is accepted for ordinary calls and provider expressions, but declarations promising exact return authority and handler targets require singleton roots.

5.3.2 Authority-preserving returns and wrappers

An exact capability-returning function may state `authority { result derives_from parameter }`; a capability member may state `authority { result derives_from Self }`. The checker requires the actual return provenance to equal that singleton source, preventing a declared authority lie.

```
capability ReadFileSystem derives_from source: capability FileSystem {
    function read(path: Text) -> Text
    effects { filesystem.read<Self> } {
        return source.read(path)
    }
}
let restricted = ReadFileSystem { source = filesystem }
```

Listing 19: Nominal single-source capability wrapper.

Implemented wrappers are closed, nominal, and have exactly one private capability source. Every operation body is checked; construction requires exactly that source; derivation cycles are rejected; the wrapper does not subtype or coerce to its source. `Self` on both sides normalizes to the original root, allowing type-changing restriction without inventing authority.

5.4 Effect Handling

The target design permits selected algebraic handlers to interpret operations and remove or transform an outward effect. They suit deterministic clocks, test randomness, tracing, state, and domain effects, but cannot conceal distribution or violate ownership.

```
capability Clock {
    function now() -> Int64 effects { clock.read<Self> }
}
capability FixedClock {
    function now() -> Int64 effects { pure }
}
function deterministic(
    clock: capability Clock,
    fixed: capability FixedClock,
) -> Int64 effects { pure } {
    return handle clock.read<clock> with fixed {
        clock.now()
    }
}
```

Listing 20: Implemented exact capability-backed clock handler.

IMPLEMENTED Exact syntax is `handle effect.name<authority> with provider { body }`. The target is one parameterized atom, resolves to exactly one module-local capability member, and that source member's complete row is exactly the matching `{ effect.name<Self> }`. The target authority must have one known root because dynamic runtime authority matching is unsupported. The provider exposes the same operation name and exact parameter/result signature with a pure row.

Provider evaluation occurs outside the handler and its effects remain outward. The provider may have multiple possible roots because provider provenance is neither matched nor subtracted. Calls are handled deeply and lexically only when the instantiated operation name and authority root exactly match. Matching atoms are subtracted from the body's outward row, while other roots, residual effects, and call-graph edges remain. The type table retains source member, provider member, and singleton target root as runtime interception metadata. The handler preserves the body's type and value.

The compiler-internal reference interpreter implements this exact deep lexical interception over runtime capability-root identity. Production runtime/backend behavior remains future work.

FUTURE WORK General algebraic handlers, resumptions, effect-row transformation, static/unparameterized targets, multiple operations, and dynamic authority matching remain target design. The v0.1 `handle clock.read with FixedClock(...)` example was not valid bootstrap syntax and is superseded above.

5.5 Typed Errors

Recoverable failure uses `Result<T, E>`. Error types are algebraic and public. `?` is type-checked early return, not hidden unwinding. Conversion requires a declared function or variant relationship.

```
function load_config(path: Path) -> Result<Config, ConfigError>
effects { filesystem.read<path.root> } {
    let bytes = filesystem.read(path).map_error(ConfigError.read_failed)?
    let syntax = config.parse(bytes).map_error(ConfigError.invalid_syntax)?
    return config.validate(syntax)?
}
```

Listing 21: Typed error propagation.

The bootstrap represents exact `Result` applications, bounded user generic instantiation, and basic `ok`/`err` propagation, but does not execute or monomorphize programs.

5.6 Panic and Unreachable States

`panic` represents a defect or violated invariant, not normal control flow. A function that may panic carries the effect unless unreachability is proved. Verified/safety profiles prohibit outward panic. Production policy may abort, terminate a task, restart a supervised actor, or trap.

```
                    unreachable because {
                        match is exhaustive after `state.validate()`
                    }
```

Listing 22: Unreachability requires an obligation.

5.7 Cancellation, Timeout, and Retry

Cancellation and timeout are typed outcomes, not invisible exceptions. Suspending operations are cancellation-aware unless noncancellable. Retry requires an `Idempotent` proof or explicit compensation.

```
let result = within 500 ms { payment_gateway.authorize(request) }
match result {
    completed(value) => value?
    timed_out => return CheckoutError.gateway_timeout
    cancelled => return CheckoutError.cancelled
}
```

Listing 23: Explicit timeout and cancellation outcomes.

5.8 Standard Effect Categories

Category	Examples
Observation	`clock.read`, `environment.read`, `configuration.read`
Nondeterminism	`random.pseudo`, `random.secure`, `scheduler.nondeterministic`
I/O	`filesystem.read/write`, `network.call`, `console.write`, `process.spawn`
State	`state.local`, `database.read/write/transaction`
Concurrency	`task.spawn/suspend`, `sync.lock`, `channel.send`, `actor.message`
Resources	`memory.allocate`, `memory.pin`, `blocking`, `gpu.dispatch`
Failure	`panic`, `cancel`, `timeout`
Boundary	`unsafe`, `ffi.call`, `host.intrinsic`

6 Contracts, Refinements, and Verification

6.1 Verification Philosophy

Sol specifications should be executable enough to test and formal enough to prove. The target compiler turns refinements, contracts, invariants, coverage, ownership facts, protocol transitions, and resource bounds into obligations. Typing, linear analysis, abstract interpretation, symbolic execution, SMT, bounded protocol model checking, or proof terms discharge different classes.

Verification must remain modular. Packages verify against dependency interfaces. Proof caches key semantic declarations, assumptions, solver configuration, and imported contracts. Formatting cannot invalidate proof; changed effects or preconditions can.

> IMPLEMENTED The bootstrap parses and resolves structured `requires` and `ensures` conditions and lowers deterministic semantic obligation templates after effect inference. It does not insert runtime checks, substitute contracts at calls, normalize solver IR, invoke SMT, or discharge proofs.

6.2 Preconditions and Postconditions

```
function withdraw(account: Account, amount: Money<USD>) -> Account
effects { pure }
requires {
    amount > 0 USD
    account.state is active
    account.balance >= amount
}
ensures {
    result.id == account.id
    result.balance == account.balance - amount
    result.state == account.state
} {
    return account with { balance = account.balance - amount }
}
```

Listing 24: Function contract with `requires`, `ensures`, `result`, and immutable update.

A caller establishes preconditions; an implementation establishes postconditions. `old(expression)` denotes entry-state values. Postconditions can select success/failure outcomes. Ownership/effects infer frames, strengthened by `preserves` clauses in the target.

6.2.1 Bootstrap contract resolution and typing

Conditions are newline- or comma-separated and clauses occur in canonical order: `effects`, `requires`, `ensures`. Each condition resolves independently in a fresh declaration-signature scope. Parameters and declarations are visible; body locals, another condition's locals, and a derived wrapper's private source are not. Ordinary expression typing applies and the predicate MUST be `Bool`.

Contract calls are allowed only after their exact target effects are finalized and pure. This includes pure functions, pure capability operations, and closed pure callbacks. Effectful calls, `return`, `?`, and handlers are rejected in predicates.

For this bootstrap rule, pure means an empty finalized observable-effect row. It does not yet prove termination, determinism, absence of mutation, or mathematical referential transparency.

In ordinary postconditions, `result` has the declared return type. Outcome prefixes are available only on declarations returning `Result<T, E>`: a `success =>` condition binds `result: T`, while `failure =>` has no result binding. A prefixed condition on any other return type is rejected with `SOL-CONTRACT-005`. Each syntactic `old(expression)` creates its own typed entry-state snapshot, even if operands repeat. `old` is limited to postconditions; nested `old` and `old(result)` are rejected.

The public deterministic template table records sequential obligation identity, owner kind/identity, clause kind, outcome, predicate and `Bool` type, result availability/type, and contiguous snapshot identity/operand/type metadata. It is deliberately a template, not proof success.

These sequential identities are deterministic for identical input but are not stable semantic identities across insertion, movement, or refactoring.

6.3 Invariants and Loops

Loops require sufficient invariants when correctness cannot be inferred. Total functions prove termination with `decreases`. Partial functions carry `diverge` and cannot serve as proofs or compile-time evaluation.

```
var index = 0
var sum = 0
while index < values.length
invariant {
    0 <= index <= values.length
    sum == values[0..index].sum()
}
decreases { values.length - index } {
    sum += values[index]
    index += 1
}
```

Listing 25: Loop invariant and termination measure.

6.4 Ghost State and Erasure

Ghost declarations exist only for specification/proof and cannot influence runtime branching, I/O, layout, or ABI unless materialized through proven reflection. The compiler erases them and checks noninterference.

```
ghost record TransferModel {
    total_before: Money<USD>
    total_after: Money<USD>
}
lemma transfer_preserves_total(
    before: Accounts, after: Accounts, amount: Money<USD>,
) -> Proof<after.total() == before.total()> {
    ...
}
```

Listing 26: Erased specification state and lemma.

6.5 Proof Modes

Mode	Behavior
`check`	Type, effect, ownership, exhaustiveness, and lightweight contracts; target policy may retain checks or warnings.
`verify`	Every declared proof obligation is discharged or explicitly assumed.
verified profile	No runtime fallback, panic, unchecked arithmetic, unverified unsafe code, or unbounded nondeterminism.
`audit`	Assumption ledger, unsafe inventory, proof coverage, solver provenance, and dependency-contract report.

Only front-end checking and semantic template construction exist today; these proof modes are target design.

6.6 Runtime Contract Fallback

External predicates may lie outside the solver fragment. `validate` retains a runtime check and yields a refined value; `assume` requires unsafe/proof-policy exception. Public APIs state static proof, dynamic validation, or boundary trust.

```
let email: EmailAddress = validate EmailAddress(input)
    .or_error(SignupError.invalid_email)?
let packet_length: PacketLength = unsafe assume_refined(raw_length)
```

Listing 27: Dynamic validation versus explicit trust.

Runtime checks are not yet generated.

6.7 Specifications and Generated Tests

`spec` blocks contain examples, properties, invariants, model functions, and generators. They compile into tests and obligations. Boundary values derive from refinements and mutation operators; generated coverage is reported separately.

```
spec normalize_username {
    example "trims and folds case" {
        input = " Alice.SMITH "
        output = "alice.smith"
    }
    property "idempotent" for all value: Text {
        normalize_username(normalize_username(value)) == normalize_username(value)
    }
    property "no surrounding whitespace" for all value: Text {
        let normalized = normalize_username(value)
        normalized == normalized.trim()
    }
}
```

Listing 28: Executable examples and universal properties.

6.8 Solver Discipline

SMT automation is constrained through typed theories, advanced-only triggers, deterministic budgets, and diagnostics naming expensive obligations. Builds record solver version/options. Timeout-sensitive heuristic proofs are unstable, never silently accepted.

- Quantifier-free arithmetic, constructors, equality, and finite maps form the preferred automatic fragment.
- Nonlinear arithmetic, unrestricted quantification, floating equivalence, and concurrency invariants may need lemmas or specialized engines.
- Obligations are named/addressable; suppression uses identity, not source line.
- Packages can bound solver time and memory per obligation and aggregate build.

All SMT and proof discharge remains future work.

6.9 Cost and Resource Contracts

Complexity and resources are specifications. Big-O claims can be checked against approved combinators/recurrences; hard byte/time limits combine static analysis and target certification. Weakened bounds are API changes.

```
function binary_search<T: Ordered>(values: SortedView<T>, target: T) -> Option<Index>
effects { pure }
cost {
    time <= logarithmic(values.length)
    allocation == 0 B
}
```

Listing 29: Complexity and allocation contract.

7 Concurrency, Protocols, Transactions, and Durability

7.1 Structured Concurrency

Tasks are lexically scoped. A parent cannot complete while children remain live unless ownership transfers to a supervisor. Errors and cancellation propagate by declared policy. Safe application code has no detached fire-and-forget task.

```
function load_dashboard(user: UserId) -> Result<Dashboard, DashboardError>
effects {
    network.call<ProfileService>
    network.call<MessageService>
    network.call<AlertService>
    task.spawn
    task.suspend
} {
    concurrent cancel_on_error {
        profile = profile_service.load(user)
        messages = message_service.recent(user)
        alerts = alert_service.active(user)
    }
    return Dashboard {
        profile = profile?
        messages = messages.or_default([])
        alerts = alerts?
    }
}
```

Listing 30: Structured concurrent fan-out.

A `concurrent` block creates a task scope and binds result handles. Policies include `cancel_on_error`, `collect_all`, and `supervise`. All results and resources must be consumed before exit.

7.2 Race Safety

Ownership prevents unsynchronized shared mutation. Task transfers require `Send`; shared immutable data requires `Share`. Synchronization wrappers expose locking effects, poisoning, fairness, and cancellation. Verified profiles may check lock order and static cycles.

7.3 Actors and Channels

Actors isolate mutable state with typed asynchronous handlers. Channels are linear endpoints whose protocol can be a state machine. Mailbox capacity and overflow policy are part of actor type because they affect reliability and bounds.

```
actor InventoryActor(capacity = 1024, overflow = reject) {
    state inventory: Map<ItemId, Stock>
    message reserve(request: ReserveRequest)
        -> Result<Reservation, ReservationError> {
        ...
    }
}
```

Listing 31: Typed actor with explicit mailbox policy.

7.4 Protocol Types and State Machines

Protocols define state-indexed resources and transitions. A transition consumes one state and returns another, rejecting invalid sequencing. Explicit boundaries can erase dynamic protocol state to runtime tags.

```
protocol Connection {
    state disconnected
    state connected(socket: Socket)
    state authenticated(socket: Socket, session: Session)
    transition connect(from: disconnected, address: Address)
        -> Result<connected, ConnectError>
    transition authenticate(from: connected, credentials: Credentials)
        -> Result<authenticated, AuthError>
    transition query(from: authenticated, request: Query)
        -> Result<(authenticated, Response), QueryError>
    transition close(from: connected | authenticated) -> disconnected
}
```

Listing 32: State-indexed connection protocol.

7.5 Transactions

`transaction` is a language-level boundary backed by provider capability. It guarantees consistent commit/rollback, resource closure, and effect restrictions. Isolation, retry, and event semantics are explicit because databases differ.

```
public transaction transfer_funds(
    source_id: AccountId,
    destination_id: AccountId,
    amount: Money<USD>,
) -> Result<TransferReceipt, TransferError>
using database = AccountsDb
isolation = serializable
retry = safe_if_conflict
effects { database.transaction<AccountsDb> clock.read } {
    ...
}
```

Listing 33: Transaction policy as part of the declaration.

A transaction may emit commit-coupled outbox events. Direct external network calls are rejected by default because rollback cannot undo them. Cross-system work uses sagas or durable workflows.

7.6 Durable Workflows and Sagas

A workflow models restart-surviving steps. Inputs, outputs, continuation state, retries, and compensation are serializable/versioned. Workflow code is deterministic relative to recorded events; direct clock, randomness, and network become durable commands.

```
workflow fulfill_order(order: OrderId) -> FulfillmentResult {
    let payment = step authorize_payment(order)
        retry exponential(max = 5)
        compensate refund_payment(payment.authorization)
    let shipment = step create_shipment(order)
        retry exponential(max = 8)
        compensate cancel_shipment(shipment.id)
    return FulfillmentResult { payment, shipment }
}
```

Listing 34: Durable saga with compensation.

7.7 Atomicity and Concurrency Verification

Advanced verified code can open named invariants in atomic sections and manipulate ghost state. Everyday code uses synchronization/protocol abstractions; the proof sublanguage isolates separation-logic reasoning for custom lock-free structures.

> **FUTURE WORK** All concurrency, protocols, transactions, workflows, ownership-based race safety, and their verification semantics in this section remain target design.

8 Modules, Schemas, Interoperability, and Runtime Model

8.1 Packages and Editions

A package contains a manifest, module tree, features, target profiles, dependency locks, and policy. Editions stabilize syntax/core semantics. A new edition may change defaults or reserve keywords but does not reinterpret older packages.

```
package "inventory-service" {
    edition = 2027
    version = "0.4.0"
    targets = [native, wasm_component]
    profile.release = verified
    dependency "sol-http" = "^0.3"
    dependency "postgres" = "^1.1"
    capabilities {
        allow network.call<PaymentGateway>
        allow database.transaction<InventoryDb>
        deny process.spawn
    }
}
```

Listing 35: Package manifest sketch.

8.2 Stable Semantic Identities

Every exported declaration receives a stable semantic ID, derived by default from package, module, kind, and evolution token. An annotation retains identity across movement/rename.

```
@stable("billing.transfer.execute.v1")
public function transfer(...) -> ... { ... }
```

Listing 36: Stable declaration identity.

Stable IDs anchor patches, API history, proof caches, documentation links, telemetry schemas, and deprecations. Reusing an ID incompatibly is rejected.

> IMPLEMENTED The bootstrap assigns every top-level declaration a versioned 128-bit ID separate from its dense session handle. The default hashes normalized module path, declaration kind, and name, so source ordering and file placement do not affect it. A named public declaration may instead retain a validated package-local `@stable` token across module movement and rename. Collisions and malformed/private/implementation uses are rejected. HIR occurrence records cover declarations, imports, resolved expression/type names, implementation trait heads, and trait bounds. Manifests and dependency identities, member/local identity, incompatible-reuse analysis beyond collisions, serialized schema guarantees, and public IR remain future work.

8.3 Canonical Typed Interpreter IR

Successful bootstrap compilation lowers to one owning, self-contained typed IR that is the sole semantic input to the compiler-internal reference interpreter. The IR has independent structural-type, declaration, callable/member, local, field, variant, expression, statement-list ID, match-arm-list ID, operand, dispatch-evidence, effect, source-file, obligation, and snapshot arenas; it does not embed or retain frontend syntax, HIR, type, effect, or contract tables. It owns compact aggregate source bytes plus per-file paths/ranges, decodes integer and the supported `\\n`, `\\r`, `\\t`, `\\\\`, and `\\\"` string escapes, orders call and constructor operands by resolved formal or field identity, represents payloadless variants directly, classifies executable calls and bound operations, retains receivers/private wrapper sources/result authority, records concrete and forwarded generic trait evidence, and points contracts at IR expression and snapshot IDs. Effect atoms own copied names and canonical local/self authority references and are sorted by those semantic values.

> IMPLEMENTED The bootstrap exposes `sol_ir_init`, `sol_ir_free`, `sol_ir_lower`, `sol_ir_lower_scoped`, and `sol_ir_validate` as C compiler-internal APIs. Lowering requires a bytewise-empty output, validates frontend pointers,

exact counts, nested type/effect/contract metadata, spans, links, owners, and slices before use, fails transactionally with a structured internal diagnostic, and validates every retained executable node and IR-native cross-reference. Bidirectional expected-type flow supports nested built-in `ok`, `err`, `some`, and `none` calls in returns, explicitly typed call/method arguments, distinct and record/variant fields, equality, propagation operands, terminal block values, and context-typed branches; ambiguous constructor uses are rejected. Propagation is limited to exact `Option` and `Result`, records its kind and `Result` residual type, and rejects user generic applications or mismatched errors. Generic inference is deliberately left-to-right: a contextual builtin is supported once earlier operands determine every required type argument; a builtin that precedes the needed evidence is diagnosed rather than inferred by global backtracking. Builtin and type-application heads are non-executable and accepted only as parts of supported call, constructor, record, or member chains; first-class heads and explicit builtin type applications are rejected. Unknown string escapes, static-authority effect atoms, and frontend constructs lacking explicit bounded lowering metadata are rejected. This IR is explicitly unstable and promises neither serialization nor public ABI compatibility.

8.3.1 Compiler-Internal Reference Interpreter

IMPLEMENTED The C17 library API in `include/sol/interpreter.h` deterministically evaluates validated owning IR after every frontend object has been freed. Sparse frames bind global IR local IDs, receivers, derived capability sources, generic substitutions, and invocation evidence. Owned runtime values cover primitives, immutable records/enums, `Option`, `Result`, distinct values, capabilities, functions, and bound operations. Execution includes checked `Int64` arithmetic, short-circuit logic, structural immutable equality, blocks/returns, matches, propagation, direct and indirect calls, explicit trait dispatch, root-preserving wrappers, host operations, and exact deep lexical handlers. Host callback results are borrowed host-owned views valid through callback return, may alias callback inputs, and are validated and deep-cloned before interpreter temporaries are released. Equality is statically restricted to primitives and recursively supported immutable aggregates; capabilities, functions, bound operations, unconstrained parameters, `Self`, and aggregates containing those values are rejected. Computed capability-operation joins retain exact executable function types through conditionals, matches, blocks, and aliases. External capabilities must have the canonical definition-specific private source chain with one preserved root; bound operations and handlers prove receiver/provider ownership and exact callable shape before host dispatch or interception. Steps, call depth, value nodes, text bytes, and host calls are independently bounded; failures use stable structured codes and bounded owned source-file paths that outlive the input IR. Named operands execute in canonical formal order because source operand order is not retained by this IR. Contracts are ignore-only: requesting checks is a structured unsupported error. This is not `sol test`, `sol run`, a VM, bytecode, code generation, contract enforcement, or a production runtime; those remain separate roadmap work.

8.4 Schema Definitions and Evolution

Persistent types carry versions and compatibility policy. Migrations are typed total functions whose loss behavior can be verified. Encoders retain unknown fields where wire formats allow, preventing destructive rolling-upgrade read/modify/write.

```
record UserProfile version 3 compatible backward {
    id: UserId @field(1)
    display_name: Text @field(2)
    locale: Locale = Locale.en_us @field(3)
    preferences: Preferences @field(4)
}
migrate UserProfile from version 2 to version 3 {
    locale = Locale.en_us
    preferences = Preferences.default()
}
```

Listing 37: Versioned schema and migration.

Field identities are independent of source ordering. Removal/change creates compatibility obligations. Database, event, and file schemas share a model but use distinct deployment adapters.

8.5 API Compatibility

The compiler computes a public semantic fingerprint over types, effects, errors, contracts, traits, layout, and protocol states. Tools classify source, binary, wire, behavior-strengthening, behavior-weakening, and breaking changes; SemVer recommendations remain owner decisions.

8.6 Foreign Function Interface

The C ABI is the universal native boundary. Headers and safe wrappers cover representable types; algebraic values use generated handles/interface records. FFI declarations include ownership, nullability, threading, blocking, errors, and effects.

```
extern "C" function zlib_compress(
    input: foreign_slice<Byte>,
    output: foreign_mut_slice<Byte>,
) -> CInt
ffi {
    ownership input = borrowed
    ownership output = borrowed_exclusive
    nullability = forbidden
    thread = any
    blocking = false
}
effects { ffi.call<zlib> unsafe }
```

Listing 38: Explicit FFI contract.

8.7 WebAssembly Components

WebAssembly components provide interface-driven composition and capability-oriented hosting. Sol maps records, enums, results, options, resources, and worlds to canonical interfaces. Effects become host imports, enabling deployment capability checks.

8.8 Runtime Profiles

Profile	Runtime model
Native static	AOT executable/library, deterministic destruction, configured allocator, optional async runtime.
Native hosted	AOT plus reflection metadata, loader, tracing, and supervised actors.
WebAssembly component	No ambient host access; imports define capabilities/resources.
Embedded	No OS assumptions, selectable allocator, interrupt-safe subset, heap-free option.
Sol VM	Portable bytecode for REPL, tests, migrations, and fast cycles; not performance reference.

8.9 ABI and Layout

Internal layout is unstable by default. `@repr(C)`, `@repr(transparent)`, and versioned stable ABI annotations opt into guarantees. Public native ABI packages freeze layout, calling convention, panic policy, and allocator ownership; the compiler emits an ABI manifest.

> **FUTURE WORK** Package manifests and dependencies, stable cross-build IDs, schema tooling, compatibility reports, FFI, WebAssembly, runtime profiles, executable IR, and ABI generation are not implemented in the bootstrap.

9 AI-Native Development and Semantic Editing

9.1 Design Objective

AI assistance is a consumer of compiler APIs, not a privileged bypass. A model proposes intent, code, proof, or patches; the compiler decides semantic validity. IDEs, refactoring tools, CI, and human automation use the same interfaces.

Text remains human-readable source of record, but edits can target declarations, parameters, effects, control-flow nodes, contracts, and schema fields by stable identity. Formatting materializes canonical source.

9.2 Intent Blocks

Intent records purpose, preservation requirements, forbidden outcomes, priorities, and assumptions. Free prose explains; enforceable fields map to contracts, lints, or review rules. Intent does not become magically executable.

```
intent transfer {
    purpose: "Move funds atomically between two accounts."
    preserve:
        - total money across both accounts
        - account identity
        - unrelated account fields
    forbid:
        - partial transfer
        - negative balance
        - transfer to the same account
    priority: correctness > observability > performance
}
```

Listing 39: Structured intent metadata.

9.3 Semantic Patch Language

The patch language operates on the typed semantic graph. It can add/remove parameters, alter effects, insert at semantic anchors, update variants, retain identity through rename, and attach preservation constraints. Ambiguous or stale targets are rejected.

```
patch billing.transfer::transfer {
    require base_fingerprint = "sha256:9e4d..."
    add parameter memo: Option<Text> after amount
    replace effect database.write<Accounts>
        with database.transaction<Accounts>
    insert before event FundsTransferred {
        audit.record_transfer_attempt(
            source = source_id, destination = destination_id, amount,
        )
    }
    preserve {
        all existing error variants
        atomicity
        behavior except optional memo support
    }
}
```

Listing 40: Semantic patch with stale-base protection.

9.4 Machine-Readable Diagnostics

Every diagnostic has a stable code, semantic subject, spans, violated rule, related declarations, proof context, and bounded repair candidates. Human prose presents this structure; repairs are constrained classes, not automatically applied guesses.

```
diagnostic {
    code: "SOL-EFFECT-014"
    severity: error
    subject: "billing.transfer.transfer"
    violation: {
        attempted_effect: "network.call<AuditService>"
        declared_effects: ["database.transaction<Accounts>", "clock.read"]
    }
    valid_repairs: [
        add_effect("network.call<AuditService>"),
        inject_capability("AuditSink"),
        move_operation_to_caller,
    ]
}
```

Listing 41: Structured effect diagnostic.

The bootstrap already emits structured human/JSON diagnostics with stable code strings and spans. Stable cross-version subject IDs and full repair schemas remain target design.

9.5 Change-Oriented Compilation

`sol check-change` compares semantic graphs and reports effects, weakened postconditions, stronger preconditions, errors, schemas, protocols, allocation, proof assumptions, and tests, not just text.

```
$ sol check-change HEAD~1..HEAD
Public behavior:
  + Transfer requests accept an optional memo.
Effects:
  ! billing.transfer.transfer now calls AuditService.
Contracts:
  = Atomicity preserved.
  = Existing errors preserved.
Cost:
  ! Worst-case network calls increased from 1 to 2.
Verification:
  842 obligations proved
  3 new obligations proved
  1 obligation unresolved:
    audit failure cannot make a committed transfer appear failed
```

Listing 42: Behavior-change report.

9.6 Context Bundles

A tool requests a bounded bundle of declarations, signatures, intent, contracts, callers/callees, effect providers, protocols, diagnostics, and recent semantic changes.

- Bundles are deterministic and hash-addressed.
- Sensitive source may be replaced with interface summaries.
- Omitted context is recorded so unsupported assumptions are visible.
- Generated edits cite subjects/base hashes for safe replay or rejection.

9.7 Trust and Approval Model

Trust does not depend on human versus AI authorship. It derives from review policy, verified properties, tests, unsafe assumptions, provenance, and signers. Policy may require approval for new capabilities, weakened contracts, unsafe blocks, schema changes, or resource changes.

9.8 Canonical Intermediate Representation

The supported semantic IR is readable, serializable, and versioned, representing resolved identities, types, effects, contracts, regions, control flow, and obligations in compact binary or canonical text. Source and IR are not isomorphic: comments/presentation remain metadata, while inferred types/effects become explicit.

```
function billing.transfer.transfer {
    stable_id "billing.transfer.execute.v1"
    input source: identity.AccountId
    input destination: identity.AccountId
    input amount: money.Money<USD> refined greater_than_zero
    output Result<TransferReceipt, TransferError>
    effects { database.transaction<Accounts> clock.read }
    obligations {
        precondition same_account_forbidden
        invariant total_balance_preserved
        postcondition receipt_matches_commit
    }
}
```

Listing 43: Human-readable Sol semantic IR sketch.

FUTURE WORK Intent declarations, patches, context bundles, change reports, policy gates, and canonical serialized Sol IR are target design. Current internal syntax/HIR/type/effect/contract tables establish executable semantics but are not a stable public IR.

10 Compiler Architecture, Toolchain, and Developer Experience

10.1 Compiler Pipeline

The target compiler consists of deterministic cacheable stages. Parsing produces lossless syntax; resolution and macro-free elaboration produce canonical AST; type/effect/ownership checking produce typed HIR; contract elaboration produces obligations; Sol IR lowers to ownership-explicit MIR and backends.

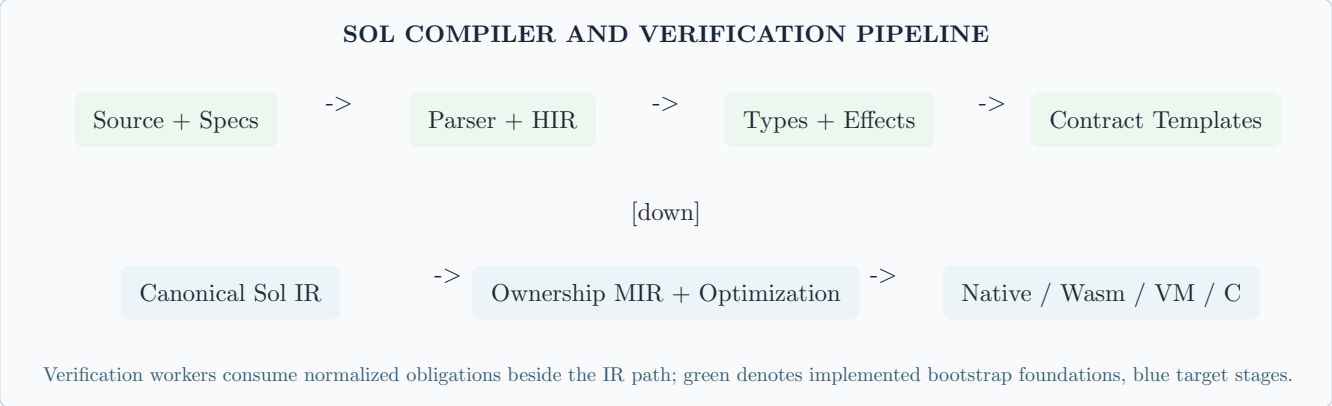

Figure 2: Target compiler and verification architecture; shaded bootstrap stages are partly executable.

10.2 Front End

The target front end provides:

- Incremental error-recovering parsing with a lossless tree for IDE/formatter use.
- Edition-aware lexing/parsing and no user token macros.
- Deterministic name resolution and explicit import graph.
- Constraint-based type/effect/region inference bounded for predictable diagnostics.
- Exhaustiveness and protocol-state checking before proof generation.

The bootstrap implements lossless tokens, recovering core parsing, token-preserving idempotent formatting, arena-backed syntax, deterministic package-session definition IDs, lexical and explicit multi-file module/import resolution, declaration-owned type resolution, typed HIR foundations, generic and nongeneric records/enums/functions/distinct/refined declarations, bounded coherent traits and constrained calls, exact invariant applications, matches, closed effects/capabilities/handlers, and generic contract/refinement templates. It does not yet implement incremental parsing, dependency packages or manifests, ownership/regions, refined construction or proof/runtime validation, richer trait or row-polymorphic generics, protocols, or width/reordering formatter policies.

10.3 Verification Engine

The target verifier normalizes contracts into typed logical IR, partitions by theory, and dispatches to analyzers/isolated solver workers with deterministic budgets. Results include assumptions, counterexamples, and hashes. Counterexamples map to source values/protocol traces. Diagnostics explain relevant paths and theories rather than merely reporting timeout.

No verifier or solver integration exists today.

10.4 Sol IR and MIR

Sol IR is high-level, typed, and effectful for tools and transforms. MIR is SSA/control flow with explicit moves, borrows, drops, regions, panic policy, and target-independent layout. Proofs attach mainly to Sol IR; memory/optimization validation attaches to MIR.

An MLIR-inspired multilevel approach can preserve Sol operations until semantics have been used while lowering to LLVM, Cranelift, WebAssembly, or embedded targets. Adopting MLIR itself is optional; textual, in-memory, and serialized forms should carry the same semantics.

10.5 Backends

Backend	Role
LLVM	Primary optimizing native AOT and link-time optimization.
Cranelift	Fast development, JIT/REPL, and simpler early code generation.
WebAssembly	Sandboxed components with capability imports and canonical interfaces.
C	Bootstrap/debug/integration path, not preferred performance/semantics reference.
Sol VM	Tests, compile-time execution, migration validation, deterministic sandboxing.

The current C17 program is only a front end and emits no executable backend output.

10.6 Incremental Compilation

Incremental keys are semantic where possible. Parsing, resolution, interfaces, monomorphization, obligations, and code generation use separate caches. A private implementation change should not invalidate downstream checking if its exported fingerprint is unchanged.

10.7 Command-Line Tool

```
sol new service inventory
sol fmt
sol check
sol verify --profile critical
sol test --derive-boundaries changed
sol run
sol build --target native --release
sol build --target wasm-component
sol check-change origin/main..HEAD
sol explain SOL-EFFECT-014
sol audit --unsafe --capabilities --proofs
sol patch apply change.solpatch
```

Listing 44: Unified target `sol` surface.

The bootstrap currently implements `sol check` and `sol fmt`. Formatting accepts one file or a recursively discovered directory package; `--check` reports noncanonical files and `--stdout` formats one file without writing.

10.8 Language Server and IDE

- Completion includes effects, capabilities, protocol states, and valid constructors.
- Editors show obligation status and counterexamples.
- Refactors are semantic patches previewed as behavior changes.
- Call hierarchy annotates effects and authority.
- Ownership views appear on request or inference failure.
- Generated tests/proof holes remain separate from production code.
- CI, IDE, and agents share a stable diagnostic protocol.

10.9 Build Reproducibility

Target manifests pin edition, package graph, target, proof engines, solver versions, build scripts, generated hashes, and capability policy. Build scripts run as WebAssembly components with declared authority rather than arbitrary host shells.

For this manual, `docs/specification.typ` is the authoritative editable source. The root `Sol_Programming_Language_Design_Specification_v0.2.pdf` is generated, never edited. With Typst 0.15.1, the offline reproducible command from the repository root is:

```
typst compile docs/specification.typ Sol_Programming_Language_Design_Specification_v0.2.pdf
```

No remote package or external asset is required. When CMake finds Typst, the optional `manual` target runs the same compiler command.

10.10 Documentation

API documentation derives from signatures, effects, errors, contracts, examples, protocols, schemas, and intent. It shows authority, suspension/allocation, recoverable failures, and proven versus runtime-checked properties.

11 Standard Library, Ecosystem, and Security

11.1 Standard Library Principles

The core library is small, stable, and freestanding. Hosted functions live in capability-oriented packages. APIs avoid hidden locale, clock, environment, filesystem, allocator, and executor dependencies. Algorithms expose complexity/allocation where practical.

11.2 Library Layers

Layer	Contents
`core`	Primitive traits, algebraic types, numeric operations, text/bytes views, ownership helpers, compile-time facilities.
`alloc`	Vectors, maps, strings, reference counting, arenas, allocator interfaces.
`std`	Filesystem, network, process, clock, random, concurrency, serialization, logging, platform abstractions.
`verify`	Propositions, proof combinators, model collections, solver theories, ghost helpers.
`component`	WebAssembly interfaces, resources, worlds, host adapters.
`embedded`	No-OS abstractions, volatile memory, interrupts, fixed-capacity collections, hardware capability traits.

11.3 Text, Time, and Randomness

Text distinguishes bytes, scalars, graphemes, normalization, and locale-sensitive behavior. Time distinguishes monotonic duration, civil time, instants, and timezone data. Randomness separates deterministic pseudo-random state from secure host entropy. These prevent convenience APIs from hiding data or nondeterminism.

11.4 Networking and Services

Network APIs are asynchronous and cancellation-aware by default. DNS, TLS roots, proxies, and credentials are capabilities. Generated service clients preserve typed errors, deadlines, idempotency, and retry rather than generic transport exceptions.

11.5 Logging, Metrics, and Tracing

Observability is an effect handled by no-op, buffered, test, or production sinks. Structured events use stable schema IDs. Pure functions cannot log; callers consume returned diagnostics or handle tracing around an effectful computation.

11.6 Package Registry and Supply Chain

- Content-addressed artifacts and reproducible source archives.
- Signed publishers and optional organization attestations.
- Machine-readable capability, unsafe, FFI, build-script, and proof metadata.
- Lockfiles pin dependency graphs and integrity hashes.
- Policy can deny new transitive capabilities or unsafe code.
- Releases publish semantic API diffs.
- Build scripts and procedural derives run in declared sandboxes.

11.7 Security Model

Sol combines memory safety, capability authority, visible effects, package sandboxing, schema validation, and auditability. Types do not eliminate logic vulnerabilities; they make authority and assumptions smaller and reviewable.

Threat	Language/tool response
Memory corruption	Safe ownership/borrowing; raw memory confined to audited unsafe and FFI.
Ambient authority	Capabilities for filesystem, network, process, environment, database, clock, randomness.
Dependency compromise	Signed/content-addressed packages, sandboxed builds, capability diffs, policy gates.

Threat	Language/tool response
Schema confusion	Nominal/field IDs, migrations, unknown-field policy, compatibility checks.
Unicode spoofing	Identifier normalization and public-API confusable detection.
AI-generated unsafe change	Policy gates on effects, unsafe, weakened contracts, and assumptions.
Denial of service	Resource contracts, bounded profiles, mailbox policy, timeout, cancellation.

11.8 Reflection and Serialization Safety

Runtime reflection is opt-in package metadata and cannot bypass invariants. Deserialization is a validation boundary with typed errors; refinements are validated or carry trusted schema proofs. Standard serialization excludes arbitrary object graphs and executable constructors.

> **FUTURE WORK** The standard library, registry, supply-chain enforcement, ownership security model, sandboxed builds, reflection, and serialization are target design. The bootstrap's capability and effect checks are executable foundations, not a complete security boundary.

12 Implementation Roadmap, Risks, and Open Questions

12.1 Recommended Implementation Strategy

Begin with a deliberately small core proving interactions among canonical syntax, algebraic types, ownership, effects, contracts, semantic identities, and diagnostics. Simultaneously building full proof automation, workflows, native targets, and an ecosystem would obscure core coherence.

12.2 Executable Bootstrap Baseline

The current C17 bootstrap provides:

- Lossless lexing, recovering parsing for core declarations/contracts/body expressions, arena syntax, deterministic package-session definition IDs, lexical and explicit multi-file module/import HIR resolution, and source-aware human/JSON diagnostics.
- Token-preserving syntax formatting with canonical whitespace, checked parse/token preservation, byte idempotence, and transactional package rewrites.
- Primitive types; exact variable-arity interned built-in/user applications; bounded generic records, enums, and free functions; constructors; exhaustive user-enum and `Bool` matching; expression/call/return checking.
- Nominal generic distinct declarations and explicit construction; refined declarations with representation-typed `self`, `Bool`/purity checking, and deterministic unresolved predicate templates.
- Nongeneric coherent traits; exact closed implementations; one inline free-function bound; symbolic bound forwarding; immediate type-directed method calls; checked method-resolution metadata and exact closed method effects.
- Closed structural function types and bounded declaration-owned callback row parameters; exact function and bound-operation effects; callback subset checking; local and per-call row inference.
- Least-fixed-point recursive inference over call-graph SCCs, including parameter and `Self` substitution.
- Capability parameters, immutable authority aliases, normalized finite may-origin sets for computed `if`/`match`, and conservative root expansion.
- Disclosure-only rows with lexical capability authority for resource effects and a closed authority-free `panic`/`diverge` subset.
- Exact authority-preserving returns and nominal checked single-source wrappers.
- Exact capability-backed handlers with syntax, source/provider matching, scoped root-sensitive subtraction, residual/provider effects, runtime metadata, and singleton target limitation.
- Structured contracts resolved in fresh signature scopes; generic template typing; `Bool` typing; finalized purity; `Result` outcome-specific `result`; distinct `old` snapshots; deterministic obligation templates.
- Versioned 128-bit top-level semantic IDs, package-local explicit evolution tokens, collision validation, and resolved declaration/import/expression/type/trait occurrence records.

It does not execute programs and is not a production compiler. In particular, general row constraints and explicit/multiple/recursive or authority-capturing effect parameters, refined construction/validation/assumptions, richer traits and constrained or higher-order generics, ownership/regions, unsafe/FFI authority gates, general algebraic handlers, static/unparameterized or runtime-dynamic handler matching, runtime contract checks, call-site contract/refinement use, logical normalization, SMT/proof discharge, concurrency, package manifests/host wiring/dependencies, member/local stable identities, width-based and declaration-reordering formatting, stable public schemas/IR, semantic patches, and code generation are not implemented.

12.3 Phased Roadmap

Phase	Scope and exit criteria
0 - Executable core	Current front-end semantics plus formalized core interactions; continue resolving contradictions with tests.
1 - Front end/interpreter	Formatter, modules, richer generics, traits, pattern matching, typed errors, effects, canonical graph, VM.
2 - Ownership/native	Affine types, borrows/regions, drop, C FFI, backend, memory-safety validation.
3 - Contracts/SMT	Runtime checks, invariants/refinements, obligation IR, solvers, counterexamples, proof cache.

Phase	Scope and exit criteria
4 - Concurrency/capabilities	Structured async, actors/channels, `Send`/`Share`, cancellation, broader handlers, sandbox hosts.
5 - Schemas/Wasm	Schema compiler, migrations, component target, bindings, compatibility reports.
6 - AI/tooling	Patches, context bundles, change reports, policy gates, agent SDK.
7 - Stabilization	Editions, registry, library hardening, security audit, performance, 1.0 RFC process.

12.4 Minimum Viable Sol

The minimum useful language must be more than "Rust with nicer syntax." Its differentiating vertical slice includes explicit public effects, executable contracts, stable graph, structured diagnostics, and semantic change reports. Required scope remains:

- Canonical formatter and editioned parser.
- Records, enums, generics, traits, `Option`, `Result`, exhaustive match.
- Affine ownership and common lexical borrowing.
- Parameterized effects, capability injection, and a decided handler subset.
- `requires`, `ensures`, `invariant`, `old`, `result`, and runtime/proof policies.
- Canonical semantic IR and stable IDs.
- JSON diagnostics and basic patch API.
- Native or WebAssembly executable output.

Several front-end parts are now implemented, but the complete minimum vertical slice has not shipped.

12.5 Major Risks

Risk	Mitigation
Feature interaction complexity	Small formal core; every surface feature lowers into it; RFC interaction matrices.
Verification brittleness	Modular contracts, stable IDs, proof caching, deterministic budgets, proof-health metrics.
Ownership ergonomics	Explainable region inference, persistent collections, scoped mutation, targeted sharing.
Effect annotation fatigue	Infer locally, normalize publicly, aliases, meaningful authority granularity.
Slow builds	Separate check/verify, semantic hashes, isolated parallel solvers, lightweight analysis first.
Model-specific tooling	Stable generic protocols usable by IDEs, linters, CI, and multiple AI systems.
Ecosystem cold start	Excellent C/Wasm interop, generated bindings, import tools, narrow high-value domain.
Unsound escape hatches	Small unsafe core, assumption ledgers, audited primitives, transformation verification.

12.6 Open Design Questions

Some v0.1 questions now have partial executable decisions:

- **Handlers:** 1.0 policy is still open, but the bootstrap has chosen exact capability-backed singleton-target handlers as a safe experimental subset. Broader resumptive/general handlers await ownership interaction work.
- **Mixed authority:** representation is decided for the bootstrap as normalized finite lexical may-origin sets with conservative expansion; runtime dynamic authority matching and long-term IR/ABI representation remain open.
- **Contract front end:** syntax, signature-scope resolution, `Bool` typing, purity, `Result`-only outcomes, snapshots, and deterministic template shape are decided. Enforcement mode, runtime fallback, logical IR, and proof discharge remain open.
- **Recursive effects:** the bootstrap decision is least-fixed-point SCC inference for omitted private closed rows. The target design of row variables and polymorphic recursive effects remains open.

Genuinely open target-design questions are:

1. How much dependent typing belongs outside ghost/proof code: compile-time naturals/protocol indices only, or a broader value-dependent fragment?
2. Is asynchronous suspension a `task.suspend` effect or a computation kind containing other effects?
3. Can protocol-state erasure and dynamic dispatch remain ergonomic without hiding runtime failure?
4. Which proof language best balances readable Sol syntax with established theorem-proving infrastructure?
5. Should native packages standardize an ABI early or rely on C/Wasm while the object model stabilizes?
6. How do semantic IDs evolve across forks, vendoring, and generated code?
7. Which resource-cost claims are portable and which require target certification?
8. Can registries enforce transitive capability policy without making legitimate platform abstractions unusable?
9. What evidence permits an AI repair to be labeled safe, behavior-preserving, or proof-preserving?
10. Should general handlers enter 1.0 after ownership is proven, or remain a later extension beyond exact capability-backed handlers?

12.7 Success Criteria

Sol succeeds if it measurably reduces maintenance defects and review effort on change-heavy systems. Evaluation includes semantic-diff accuracy, effect-leak prevention, contract coverage, repair success, proof stability, refactor reliability, onboarding, and repository context required for a correct change, not only speed or line count.

13 Worked Reference Examples

These examples preserve the v0.1 reference corpus. Unless explicitly stated, they express target design and are not complete bootstrap conformance tests.

13.1 HTTP Handler with Explicit Authority

```
module api.accounts
use http.{Request, Response, Status}
use identity.AccountId
public function get_account(
    request: Request,
    repository: capability AccountRepository,
    authorization: capability Authorization,
) -> Result<Response, HandlerError>
effects {
    database.read<repository>
    authorization.check<authorization>
    tracing.emit
} {
    let account_id = request.path.parameter("account_id")
        .parse<AccountId>().map_error(HandlerError.invalid_account_id)?
    authorization.require(
        request.principal, Permission.read_account(account_id),
    )?
    let account = repository.find(account_id)?
    return match account {
        some(value) => Response.json(Status.ok, value.public_view())
        none => Response.empty(Status.not_found)
    }
}
```

Listing 45: Service handler without ambient repository or authorization access.

The signature discovers operational authority. Tests replace both capabilities. Policy can reject network or write effects. A cache wrapper can require proof that authorization precedes protected cached data.

13.2 Idempotent Inventory Reservation

```
intent reserve_inventory {
    preserve:
        - available stock never becomes negative
        - retries do not reserve twice
        - stock update and reservation record commit atomically
}
public transaction reserve(order: OrderId, item: ItemId, quantity: Quantity)
    -> Result<Reservation, ReservationError>
using database = InventoryDb
isolation = serializable
effects { database.transaction<InventoryDb> clock.read }
ensures {
    success => reservations.contains(order, item)
    success => inventory[item].available
        == old(inventory[item].available) - quantity
    failure => inventory[item].available == old(inventory[item].available)
} {
    match reservations.find(order, item) {
        some(existing) => return existing
        none => continue
    }
    let stock = inventory.lock(item)?
    require stock.available >= quantity else {
        return ReservationError.insufficient_stock(
            requested = quantity, available = stock.available,
        )
    }
    stock.available -= quantity
    let reservation = Reservation { order, item, quantity, created_at = clock.now() }
    reservations.insert(reservation)
    emit commit InventoryReserved(reservation)
    return reservation
}
spec reserve {
    property "retry is idempotent" for all valid request {
        let first = reserve(request)
        let second = reserve(request)
        expect second == first
        expect inventory_change(second_call) == 0
    }
}
```

Listing 46: Transaction, idempotency intent, postconditions, and property.

13.3 Embedded Controller

```
record ControllerState { integral: Float32, previous_error: Float32 }
function control_tick(
    state: inout ControllerState,
    target: Celsius,
    measured: Celsius,
    dt: Second,
) -> DutyCycle
effects { pure }
requires { 0 s < dt <= 100 ms }
ensures { 0 percent <= result <= 100 percent }
resources {
    profile = real_time
    stack <= 512 B
    heap == 0 B
    worst_case_time <= 50 us
} {
    let error = target - measured
    state.integral = clamp(
        state.integral + error.value * dt.value, -1000.0, 1000.0,
    )
    let derivative = (error.value - state.previous_error) / dt.value
    state.previous_error = error.value
    return DutyCycle.clamped(
        kp * error.value + ki * state.integral + kd * derivative,
    )
}
```

Listing 47: Heap-free real-time function with units and bounded result.

13.4 Protocol-Typed Database Session

```
protocol Session {
    state disconnected
    state connected(connection: DbConnection)
    state transaction_open(connection: DbConnection, tx: Tx)
    transition connect(from: disconnected, config: DbConfig)
        -> Result<connected, ConnectError>
    transition begin(from: connected)
        -> Result<transaction_open, BeginError>
    transition commit(from: transaction_open)
        -> Result<connected, CommitError>
    transition rollback(from: transaction_open) -> connected
    transition close(from: connected) -> disconnected
}
using session = Session.connect(config)?
using tx = session.begin()?
repository.update(tx, record)?
session = tx.commit()?
```

Listing 48: Legal lifecycle encoded in resource state.

13.5 Semantic Refactor

```
patch package inventory-service {
    select calls to legacy_clock.now
    replace with capability clock.now
    add capability parameter clock: capability Clock
        to nearest public entrypoint
    thread parameter through private callers
    preserve {
        returned timestamps
        error variants
        allocation behavior
    }
    require {
        no remaining effect environment.read<TZ>
        all tests use explicit clock handlers
    }
}
```

Listing 49: Cross-cutting refactor expressed by meaning, not search.

Appendix A. Syntax and Grammar Sketch

This appendix is illustrative, not parser-complete normative grammar. The authoritative edition grammar should be generated from compiler source. The sketch establishes regularity and reveals syntax interactions. It is updated with executable-core handler and authority forms.

```
source_file := module_decl edition_decl? use_decl* declaration*
module_decl := "module" module_path NEWLINE
edition_decl := "edition" INTEGER NEWLINE
use_decl := "use" use_path NEWLINE
declaration := visibility? annotation* (
    record_decl | enum_decl | type_decl | trait_decl | implementation_decl
    | capability_decl
    | function_decl | protocol_decl | transaction_decl | workflow_decl
    | intent_decl | spec_decl | test_decl
)
function_decl := "function" qualified_name function_generic_params?
    "(" parameter_list? ")" "->" type
    authority_clause? effect_clause? requires_clause? ensures_clause?
    cost_clause? resource_clause? block
effect_clause := "effects" "{" effect_entry* "}"
type_generic_params := "<" TYPE_NAME ("," TYPE_NAME)* ">"
function_generic_params := "<" TYPE_NAME (":" TYPE_NAME)?
    ("," TYPE_NAME (":" TYPE_NAME)?)*
    ("," "effects" TYPE_NAME)? ">" | "<" "effects" TYPE_NAME ">"
authority_clause := "authority" "{" "result" "derives_from"
    (IDENTIFIER | "Self") "}"
requires_clause := "requires" "{" contract_condition* "}"
ensures_clause := "ensures" "{" contract_condition* "}"
contract_condition := outcome? expression
outcome := ("success" | "failure") "=>"
record_decl := "record" TYPE_NAME type_generic_params? version_clause? "{" field_decl* "}"
enum_decl := "open"? "enum" TYPE_NAME type_generic_params? "{" variant_decl* "}"
trait_decl := "trait" TYPE_NAME "{" trait_method* "}"
implementation_decl := "implementation" TYPE_NAME "for" type
    "{" implementation_method* "}"
trait_method := "function" IDENTIFIER "(" parameter_list? ")" "->" type
    effect_clause
implementation_method := "function" IDENTIFIER
    "(" parameter_list? ")" "->" type effect_clause block
capability_decl := "capability" TYPE_NAME
    ("derives_from" IDENTIFIER ":" "capability" TYPE_NAME)?
    "{" capability_member* "}"
type_decl := "type" TYPE_NAME type_generic_params? "="
    ("distinct" type | "refined" type refinement)
statement := let_stmt | var_stmt | assignment | return_stmt
    | require_stmt | emit_stmt | using_stmt | expression_stmt
expression := literal | path | call | block | if_expr | match_expr
    | binary_expr | unary_expr | record_expr | type_apply_expr
    | lambda_expr | handle_expr
type_apply_expr := (path | field_expr) "<" type_list ">"
    # followed by "(" or "." or "{"
handle_expr := "handle" effect_name "<" expression ">"
    "with" expression block
match_expr := "match" expression "{" match_arm+ "}"
match_arm := pattern guard? "=>" expression
result_type := "Result" "<" type "," type ">"
option_type := "Option" "<" type ">"
function_type := "function" "(" type_list? ")" "->" type
    (effect_clause | "effects" TYPE_NAME)
```

Listing 50: Condensed grammar sketch.

In canonical bootstrap source, `authority` precedes `effects`, followed by `requires` and `ensures`; each clause occurs at most once. Newlines or commas separate contract conditions. `success` and `failure` are valid only in `ensures` on declarations returning `Result<T, E>`; `result` and `old` are contextual contract forms, and `old` is valid only in postconditions. A free-function authority source names a direct capability parameter, while a capability member uses `Self`. The bounded bootstrap permits one inline trait bound per free-function type parameter and one trailing `effects E` parameter; callback types use `effects E`, while the declaration row names `E` inside braces. Trait and implementation methods begin with `self: Self`, require explicit closed effects, and do not admit authority or contract clauses. Exact handlers always have an explicitly parameterized target and cannot transform an unresolved row. In expressions, a balanced `<...>` is recognized as type arguments only on a path/field-like head when it closes before `(`, `.`, or `{`; ordinary comparisons remain binary expressions.

Appendix B. Standard Effect Taxonomy

Concrete effects are parameterized by capabilities, resources, or roots. Aliases may group domain effects but expand to canonical rows in public semantic IR. In the bootstrap, declaration rows are closed except while symbolically checking the bounded callback-driven row parameter; every concrete call instantiation closes that row. Parameter- and `Self`-dependent atoms expand over normalized may-origin roots but cannot be captured by a row argument.

Effect	Meaning and notes
`pure`	Empty observable row; deterministic for equal inputs; total unless `diverge`.
`diverge`	May not terminate; excluded from proofs/compile-time evaluation.
`panic`	Abnormal termination due to violated invariant.
`memory.allocate`	Allocation through a specified allocator/region.
`memory.pin`	Address stability constraining movement/collection.
`clock.read`	Wall, civil, or monotonic clock capability.
`random.pseudo`	Deterministic pseudo-random state.
`random.secure`	Secure host entropy capability.
`filesystem.read/write`	Filesystem-root-constrained access.
`network.call/listen`	Named outbound service/inbound endpoint class.
`database.read/write/transaction`	Repository/database capability operations.
`console.read/write`	Interactive terminal or stream.
`environment.read`	Host environment; discouraged outside wiring.
`process.spawn`	Host process creation/control.
`task.spawn/suspend`	Child creation or yielding/suspension.
`sync.lock`	Synchronization acquisition.
`channel.send/receive`	Typed endpoint use.
`tracing.emit`	Structured observability event.
`unsafe`	Manually established memory/type invariants.
`ffi.call`	Foreign ABI boundary; safe generated wrappers may encapsulate it.

Appendix C. Diagnostic and Patch Schemas

C.1 Diagnostic Envelope

```
{
  "schema": "sol.diagnostic/1",
  "code": "SOL-EFFECT-014",
  "severity": "error",
  "message": "undeclared effect network.call<AuditService>",
  "subject": {
    "stable_id": "billing.transfer.execute.v1",
    "kind": "function"
  },
  "locations": [{
    "file": "billing/transfer.sol",
    "start": { "line": 42, "column": 9 },
    "end": { "line": 42, "column": 31 },
    "role": "primary"
  }],
  "facts": {
    "attempted_effect": "network.call<AuditService>",
    "declared_effects": ["database.transaction<Accounts>", "clock.read"]
  },
  "repairs": [
    {
      "kind": "add_effect",
      "safety": "behavior_expanding",
      "requires_approval": "capability_policy"
    },
    { "kind": "inject_capability", "safety": "architecture_change" }
  ]
}
```

Listing 51: Proposed diagnostic JSON envelope.

C.2 Patch Envelope

```
{
  "schema": "sol.semantic-patch/1",
  "base_graph": "sha256:...",
  "operations": [{
    "op": "add_parameter",
    "target": "billing.transfer.execute.v1",
    "after": "amount",
    "parameter": { "name": "memo", "type": "Option<Text>" }
  }],
  "preserve": ["public_errors", "atomicity", "existing_success_behavior"],
  "author": {
    "kind": "tool",
    "name": "assistant",
    "signature": "optional-attestation"
  }
}
```

Listing 52: Proposed semantic patch exchange format.

Appendix D. Comparative Positioning

Sol is intentionally synthetic. No current language provides the complete combination proposed here. This comparison describes influence, not direct compatibility.

System	Relevant strengths	What Sol adds or changes
F* + Pulse	Dependent/refinement types, effects, SMT, ghost erasure, mutable-state/concurrency proofs.	Conventional systems surface, semantic editing, progressive modes, change reports, unified toolchain.
Dafny	Contracts, ghost code, automated verification, familiar imperative syntax.	Ownership, general effects, native execution, capabilities, patch protocols.
Rust	Ownership, borrowing, enums, exhaustive matching, typed errors, systems ecosystem.	Native effects/contracts, proofs, protocols, schemas, semantic interfaces.
Koka	Precise effect inference, algebraic handlers, effect polymorphism.	Ownership, nominal systems types, contracts, transactions, schemas, deployment profiles.
Idris 2	Dependent/quantitative types, linear resources, protocol examples.	SMT-first routine contracts, effects, maintenance tooling, less-dependent everyday language.
SPARK	Contracts, type predicates, absence-of-runtime-error proof, assurance discipline.	ADTs, ownership ergonomics, effect/capability rows, modern concurrency, agent tooling.
Pony	Actors and reference capabilities for race-safe sharing.	General ownership, proofs, typed errors, transactions, schemas, broader backends.
MLIR	Multilevel IR, textual/in-memory/serialized forms, reusable transforms.	Language-specific stable graph with contracts, effects, ownership, and patch identity.

D.1 Closest Current Language

For the complete conceptual design, F* with Pulse is closest because it combines dependent/refinement types, effectful programming, automated proof, ghost erasure, mutable state, and concurrent separation logic. Dafny is closest to the readable contract surface; Rust to production implementation substrate; Koka to target effect rows and broader handler model.

Sol's differentiating claim is integration of safe implementation, explicit authority, progressive proof, stable semantic representation, and change-oriented tooling. The bootstrap does not yet realize that complete comparison; it tests a narrow executable core.

Appendix E. References and Design Influences

These primary sources informed comparison and implementation discussion; they are not normative dependencies.

- [R1] F* - A Proof-Oriented Programming Language: dependent typing, effects, SMT/tactics.
- [R2] F* Tutorial, Computational Effects: computation types, user effects, divergence, ghost computation.
- [R3] Pulse: proof-oriented mutable state, concurrency, specifications, and separation logic.
- [R4] Dafny Documentation and Reference Manual: contracts, framing, static verification, ghost features.
- [R5] The Koka Programming Language: effect inference, row polymorphism, handlers, pure/effectful types.
- [R6] The Rust Programming Language, Understanding Ownership: ownership and borrowing without GC.
- [R7] Idris 2 Documentation, Linear Resources: quantitative resources and state-indexed protocols.
- [R8] SPARK User's Guide, Subprogram and Type Contracts: high-assurance contract checking.
- [R9] Pony Tutorial, Reference Capabilities: concurrency-safe sharing and transfer.
- [R10] MLIR Language Reference and Rationale: multilevel textual/in-memory/serialized IR.
- [R11] Cranelift: code generator and CLIF IR for development and JIT-style execution.
- [R12] WebAssembly Component Model: composable components and rich cross-language interfaces.

Closing Statement

Sol is a proposal for a new interface between intent, implementation, verification, and change. Its syntax is conservative. Its novelty is making effects, authority, contracts, resource behavior, semantic identity, and change consequences normal programming rather than layers reconstructed afterward.

The most important experiment is not whether Sol compiles a benchmark. It is whether a human or model can modify a nontrivial system with less hidden context, receive bounded meaningful obligations, and demonstrate mechanically or operationally that requested behavior changed while unrelated behavior did not.

RECOMMENDED NEXT ARTIFACTS Continue decomposing this specification into focused RFCs: Core Syntax and ADTs; Closed Effects and Capability Provenance; Exact Handlers; Ownership and Regions; Contracts and Obligation IR; Semantic Graph and Stable IDs; Structured Diagnostics; and Semantic Patch Protocol. Each RFC should identify target semantics, executable bootstrap behavior, and deliberate future work.

End of Design Specification v0.2